Перечень артефактов и протоколов программного проекта

Выполнил: Моисеев Андрей
Группа: 23Ис-41
Дата:

---

1. Цель работы

Научиться определять, какие артефакты и протоколы необходимы для конкретного программного проекта, обосновывать их выбор и оформлять в виде структурированного перечня. Закрепить понимание того, что проект — это не только код, но и совокупность документов, соглашений и правил взаимодействия между участниками.

---

2. Описание проекта

Название проекта: Платформа маркетплейса «Wildberries» (модель проекта на основе публично известной архитектуры и процессов компании).

Цель проекта: обеспечить работу онлайн-площадки, где продавцы размещают товары, а покупатели находят, заказывают и получают их через сеть пунктов выдачи и логистическую инфраструктуру.

Целевая аудитория:

- покупатели (физические лица), использующие сайт и мобильное приложение;
- продавцы (партнёры), управляющие товарами и заказами через личный кабинет;
- сотрудники пунктов выдачи заказов (ПВЗ);
- внутренние операционные команды (логистика, поддержка, аналитика).

Ключевые функции (минимум 5):

1. Каталог и поиск товаров с фильтрацией и ранжированием.
2. Корзина и оформление заказа с оплатой.
3. Управление заказами и статусами доставки (OMS).
4. Личный кабинет продавца: загрузка карточек, цены, остатки.
5. Управление логистикой: маршрутизация, склады, ПВЗ.
6. Система отзывов и рейтингов.
7. Уведомления и коммуникация с пользователем.

Участники проекта:

- продакт-менеджер (владелец продукта);
- системный/бизнес-аналитик;
- архитектор;
- команды разработки (backend, frontend, мобильная разработка);
- QA-инженеры;
- технические писатели;
- DevOps/SRE;
- дизайнеры интерфейсов;
- представители бизнеса (категорийные менеджеры, логистика).

---

3. Перечень артефактов

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
1	Продуктовые требования (PRD)	Документ	Фиксирует бизнес-цели, сценарии пользователей и критерии успеха фичи	Продакт-менеджер	Команда разработки, бизнес	Анализ требований	Confluence / Wiki
2	Архитектурная схема (C4, UML)	Схема	Описывает микросервисную архитектуру,	Архитектор / техписатель	Разработчики, DevOps, новые	Проектирование	draw.io / PlantUML / Wiki

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
			границы сервисов и взаимодействия		сотрудники		
3	Спецификация API (OpenAPI/Swagger)	Документ / контракт	Описывает контракты REST/gRPC между сервисами и для внешних партнёров	Backend-разработчик / техписатель	Разработчики, партнёры, QA	Проектирование / реализация	Репозиторий / Swagger UI
4	ER-диаграмма базы данных	Схема	Показывает структуру таблиц, связей и ключей (PostgreSQL)	Backend-разработчик	Разработчики, аналитики, техписатель	Проектирование	drawSQL / Wiki
5	Исходный код сервисов (Go, др.)	Код	Реализует бизнес-логику микросервисов	Команды разработки	Разработчики, QA, DevOps	Реализация	Git-репозиторий

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
			(каталог, заказы, оплата и т.д.)				
6	Инфраструктурные конфигурации (K8s, Docker)	Конфигурация	Описывают развёртывание сервисов, окружения и параметры кластера	DevOps / SRE	DevOps, разработчики	Реализация / развёртывание	Git-репозиторий
7	Тест-кейсы и чек-листы	Тест-кейс	Проверяют функциональность и интеграции (API, UI, логистика)	QA-инженер	QA, разработчики	Тестирование	TestRail / трекер
8	Отчёт о тестировании	Документ	Фиксирует результаты прогонов,	QA-инженер	Команда, продакт	Тестирование	Трекер / Wiki

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
			дефекты и готовность к релизу				
9	Runbook / инструкция по эксплуатации	Документ	Описывает действия при инцидентах, деплое и мониторинге	DevOps / SRE	Дежурные, DevOps	Эксплуатация	Wiki (Markdown)
10	Документация для продавцов (API партнёра)	Документ	Описывает методы загрузки карточек, цен, остатков и работу с заказами	Техписатель / backend	Продавцы-партнёры	Внедрение / эксплуатация	Портал документации
11	Пользовательская документация (справка)	Документ	Объясняет покупателям и сотрудникам ПВЗ	Техписатель	Пользователи, поддержка	Внедрение / эксплуатация	Веб-страница / PDF

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
			работу с сервисом				
12	ADR (Architecture Decision Records)	Документ	Фиксирует принятые архитектурные решения и их обоснование	Архитектор / тимлид	Разработчики, архитекторы	Все этапы	Репозиторий (Markdown)

---

4. Перечень протоколов

№	Протокол	Назначение	Участники	Периодичность	Форма фиксации
1	Протокол груминга / планирования бэклога	Согласование приоритетов, оценка задач с учётом особенностей кода и кросс-командных зависимостей	Продакт, тимлид, разработчики, QA	Еженедельно / по спринту	Запись в Jira / Confluence

№	Протокол	Назначение	Участники	Периодичность	Форма фиксации
2	Протокол код-ревью	Определяет правила проверки кода перед merge (стиль, безопасность, тесты)	Разработчики, тимлид	По событию (каждый pull request)	Комментарии в GitLab / GitHub
3	Протокол согласования API-контрактов	Регулирует утверждение изменений в OpenAPI/gRPC-контрактах между командами	Backend-разработчики, архитектор, потребители API	По событию	Merge request + запись в Wiki
4	Протокол тестирования и приёмки фичи	Фиксирует порядок прогона тестов, критерии готовности и приёмку продактом	QA, разработчики, продакт	По событию (перед релизом)	Отчёт в TestRail / Jira
5	Протокол управления изменениями (Change Management)	Регулирует внесение изменений в требования и архитектуру	Продакт, аналитик, тимлид, архитектор	По событию	Заявка в Jira / RFC-документ

№	Протокол	Назначение	Участники	Периодичность	Форма фиксации
		, оценку влияния на сроки			
6	Протокол инцидент-менеджмента	Определяет порядок действий при сбоях, эскалации и постмортема	DevOps/SRE, дежурные, тимлид	По событию	Инцидент-тикет + постмортем в Wiki
7	Протокол обмена данными с внешними системами (платёжные, логистические)	Описывает форматы, частоту, ответственность и обработку ошибок при интеграциях	Backend, DevOps, представители внешних систем	По событию (при настройке/изменении)	Спецификация / документ

---

5. Связь артефактов и протоколов

1. **Протокол груминга связан с артефактом «Продуктовые требования (PRD)».** На груминге команда уточняет требования, выявляет скрытые зависимости (например, несколько реализаций экрана из-за A/B-тестов) и корректирует оценку. Без этого протокола задачи попадают в спринт с нереалистичными сроками .
2. **Протокол согласования API-контрактов регулирует артефакт «Спецификация API».** Любое изменение контракта должно быть проверено потребителями (другими командами или партнёрами), иначе интеграции ломаются без предупреждения.

3. **Протокол код-ревью связан с артефактом «Исходный код сервисов».** Он задаёт правила, по которым код проверяется перед попаданием в основную ветку, что особенно важно в микросервисной архитектуре с множеством команд.
4. **Протокол тестирования и приёмки фичи связан с артефактами «Тест-кейсы» и «Отчёт о тестировании».** Протокол определяет, какие проверки обязательны и как фиксируется результат, а тест-кейсы и отчёт являются его выходными артефактами.
5. **Протокол инцидент-менеджмента связан с артефактом «Runbook / инструкция по эксплуатации».** Runbook описывает действия при сбоях, а протокол инцидент-менеджмента регулирует, кто и как эти действия выполняет, как эскалирует и как оформляет постмортем.

---

6. Обоснование выбора

Обязательные артефакты для любого проекта: PRD (или ТЗ), архитектурная схема, исходный код, тест-кейсы, отчёт о тестировании, runbook, протоколы код-ревью и управления изменениями. Они обеспечивают понимание целей, контроль качества и воспроизводимость эксплуатации.

Специфичные для проекта Wildberries артефакты: спецификация API для продавцов-партнёров, ER-диаграмма шардированной БД, ADR для микросервисной архитектуры, документация для ПВЗ. Они обусловлены масштабом платформы, наличием внешних партнёров и сложной логистической инфраструктурой. В компании отдельно выделяют роль технического писателя, который отвечает за актуализацию архитектурной документации, UML-схем и API-спецификаций .

Критичные протоколы для проекта: протокол груминга (из-за кросс-командных зависимостей и A/B-тестов, которые удваивают объём работ) , протокол согласования API-контрактов (множество сервисов взаимодействуют через gRPC/REST), протокол инцидент-менеджмента (высокая нагрузка и критичность доступности), протокол управления изменениями.

Что произойдёт при отсутствии артефакта или протокола:

- без PRD и согласования требований — команда реализует не то, что нужно бизнесу;
- без спецификации API — интеграции между сервисами и с партнёрами ломаются при изменениях;
- без протокола груминга — оценки сроков становятся нереалистичными, TTM растёт ;

- без runbook и протокола инцидент-менеджмента — время восстановления после сбоев увеличивается;
- без код-ревью — качество кода падает, растёт технический долг.

---

7. Вывод

В ходе работы был проанализирован проект масштабной платформы маркетплейса на примере Wildberries. Описаны цель, аудитория, функции и участники, составлены перечни артефактов (12 позиций) и протоколов (7 позиций), показаны связи между ними и обоснован выбор. Работа демонстрирует, что в крупном проекте артефакты и протоколы не существуют изолированно: они образуют систему, где каждый элемент снижает риски — от неверных требований до сбоев в продакшене.